

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 6

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement : Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm :

1. Import the required libraries such as TensorFlow, Keras, NumPy, Matplotlib, and OS.
2. Specify and verify the path of the Tomato-Village training dataset and identify the six disease classes.
3. Count and visualize the images belonging to each class.
4. Resize the input images to 160×160 pixels.
5. Divide the dataset into training and validation subsets.
6. Apply data augmentation using horizontal flipping, rotation, zooming, and contrast adjustment.
7. Normalize the image pixel values to the range $[0, 1]$.
8. Construct a CNN with four convolutional layers containing 32, 64, 128, and 256 filters, followed by max-pooling layers.
9. Add Global Average Pooling, a 128-neuron Dense layer, Dropout (0.4), and a six-neuron Softmax output layer.
10. Compile the CNN using the Adam optimizer, sparse categorical cross-entropy loss, and accuracy metric.
11. Train the CNN for a maximum of 18 epochs with Early Stopping while recording training and validation performance.
12. Plot the Training vs. Validation Accuracy and Training vs. Validation Loss graphs.

13. Evaluate the trained CNN on the validation dataset and calculate the final validation accuracy and validation loss.
14. Display the final evaluation results and Stop.

Code :

```
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from tensorflow.keras import layers, models
from sklearn.metrics import classification_report, confusion_matrix

DATASET_PATH = r"C:\Users\Akshada\OneDrive\Documents\DL Lab\train"
print("Dataset Path:", DATASET_PATH)
if os.path.exists(DATASET_PATH):
    print("Dataset found successfully!")
else:
    print("Dataset path not found.")
#find the classes
classes = sorted([
    folder for folder in os.listdir(DATASET_PATH)
    if os.path.isdir(os.path.join(DATASET_PATH, folder))
])

print("Classes found:\n")
for i, class_name in enumerate(classes):
    print(i, ":", class_name)

print("\nTotal Classes:", len(classes))

print("Images per class:\n")
class_counts = {}
for class_name in classes:
    class_path = os.path.join(
        DATASET_PATH,
        class_name
    )
    count = len([
```

```

        file for file in os.listdir(class_path)
        if file.lower().endswith(
            ".jpg", ".jpeg", ".png")
        )
    ]
    class_counts[class_name] = count
    print(
        f"{class_name}: {count}"
    )

IMG_SIZE = (160, 160)
BATCH_SIZE = 24
VALIDATION_SPLIT = 0.15
SEED = 42
EPOCHS = 18
NUM_CLASSES = 6

train_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_PATH,
    validation_split=VALIDATION_SPLIT,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int"
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_PATH,
    validation_split=VALIDATION_SPLIT,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int"
)
class_names = train_ds.class_names
print("Class mapping:\n")
for index, name in enumerate(class_names):
    print(
        index,
        "→",
        name
    )

plt.figure(figsize=(10, 8))

```

```

for images, labels in train_ds.take(1):
    for i in range(8):
        plt.subplot(2, 4, i + 1)
        plt.imshow(
            images[i].numpy().astype("uint8"))
        plt.title(
            class_names[labels[i]])
        plt.axis("off")
plt.tight_layout()
plt.show()

```

```

AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(
    AUTOTUNE
)
val_ds = val_ds.prefetch(
    AUTOTUNE
)

```

```

Model
model = models.Sequential([
    layers.Input(
        shape=(160, 160, 3)
    ),
    layers.Rescaling(
        1.0 / 255
    ),
    augmentation,
    layers.Conv2D(
        32,
        (3, 3),
        padding="same",
        activation="relu"
    ),
    layers.MaxPooling2D(
        (2, 2)
    ),
    layers.Conv2D(
        64,
        (3, 3),
        padding="same",
        activation="relu"
    ),

```

```

    ),
    layers.MaxPooling2D(
        (2, 2)
    ),
    layers.Conv2D(
        128,
        (3, 3),
        padding="same",
        activation="relu"
    ),

    layers.MaxPooling2D(
        (2, 2)
    ),

    layers.Conv2D(
        256,
        (3, 3),
        padding="same",
        activation="relu"
    ),
    layers.MaxPooling2D(
        (2, 2)
    ),
    layers.GlobalAveragePooling2D(),
    layers.Dense(
        128,
        activation="relu"
    ),
    layers.Dropout(
        0.4
    ),
    layers.Dense(
        NUM_CLASSES,
        activation="softmax"
    )
])
optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.001
)
model.compile(
    optimizer=optimizer,
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

```

```

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=4,
    restore_best_weights=True
)
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    callbacks=[
        early_stopping
    ]
)

# graphs
plt.figure(figsize=(8, 5))
plt.plot(
    history.history["accuracy"],
    marker="o",
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    marker="o",
    label="Validation Accuracy"
)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title(
    "CNN Training vs Validation Accuracy"
)
plt.legend()
plt.grid()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(
    history.history["loss"],
    marker="o",
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    marker="o",

```

```
        label="Validation Loss"
    )
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title(
    "CNN Training vs Validation Loss")
plt.legend()
plt.grid()
plt.show()

val_loss, val_accuracy = model.evaluate(
    val_ds,
    verbose=1
)
print(
    f"Validation Accuracy: "
    f"{val_accuracy * 100:.2f}%"
)
print(
    f"Validation Loss: "
    f"{val_loss:.4f}"
)
```

Output :

- **Distinct classes**

Classes found:

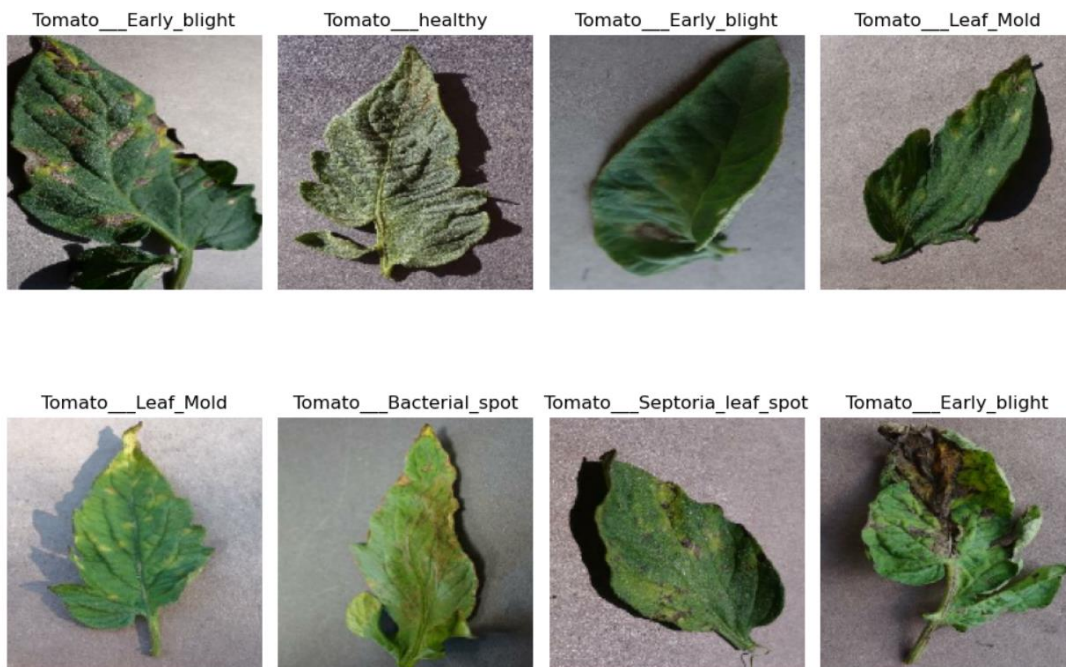
```
0 : Tomato__Bacterial_spot
1 : Tomato__Early_blight
2 : Tomato__Late_blight
3 : Tomato__Leaf_Mold
4 : Tomato__Septoria_leaf_spot
5 : Tomato__healthy
```

Total Classes: 6

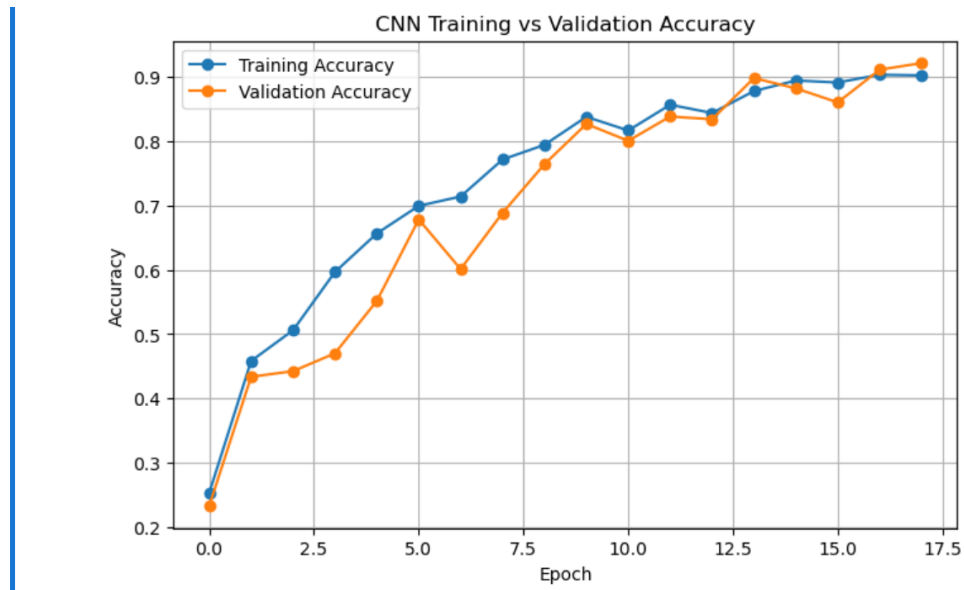
Images per class:

```
Tomato__Bacterial_spot: 768
Tomato__Early_blight: 762
Tomato__Late_blight: 763
Tomato__Leaf_Mold: 760
Tomato__Septoria_leaf_spot: 768
Tomato__healthy: 765
```

- **Samples of Different Tomato leaves dataset**



- **Training vs Validation Accuracy visualization:**



- **Training vs Validation Loss Visualization**

